Spring Security Database Authentication: Internals and Complete Login Flow

Spring Security performs authentication before the request reaches the controller. For database authentication, it connects the incoming username and password with our database through a set of filters, authentication abstractions, and user-loading contracts.

This chapter explains how those components fit together and how to implement the complete flow in a Spring Boot application.

---

1. Where Spring Security Works

Spring Security's servlet support is built on servlet filters.

```
HTTP Request
    ↓
Servlet Filters
    ↓
Spring Security Filters
    ↓
DispatcherServlet
    ↓
Controller
```

Security must run before the controller because the application first needs to determine:

- Who is making the request?
- Is the supplied identity valid?
- What roles or authorities does the user have?
- Is the user allowed to access the requested resource?

A simplified security-filter order looks like this:

```
Security context handling
        ↓
Exploit protection, such as CSRF
        ↓
Authentication filters
        ↓
Anonymous authentication
        ↓
Exception translation
        ↓
Authorization
```

The real filter chain can contain additional filters. The important principle is:

```
Protection → Authentication → Exception Handling → Authorization
```

Authentication must happen before authorization. Spring Security cannot decide whether a user has `ROLE_ADMIN` until it has first established who that user is.

---

2. `DelegatingFilterProxy`

`DelegatingFilterProxy` is a real servlet filter known to the servlet container. It does not implement all of Spring Security itself. Instead, it finds a filter bean managed by Spring and delegates the request to it.

Conceptually:

```
public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain
) {
    Filter springFilter = findFilterBeanInSpring();
    springFilter.doFilter(request, response, chain);
}
```

The connection is:

```
Tomcat
   ↓
DelegatingFilterProxy
   ↓
FilterChainProxy
```

> `DelegatingFilterProxy` connects the servlet-container lifecycle with Spring-managed filter beans.

---

3. `FilterChainProxy`

`FilterChainProxy` is the central filter of Spring Security's servlet architecture. It stands in front of one or more `SecurityFilterChain` objects.

```
FilterChainProxy
    ├── SecurityFilterChain for /api/**
    ├── SecurityFilterChain for /admin/**
    └── SecurityFilterChain for every other request
```

For every request, `FilterChainProxy` asks:

> Which `SecurityFilterChain` should process this request?

It then executes the filters belonging to the selected chain.

---

4. `SecurityFilterChain`

A `SecurityFilterChain` contains two things:

```
A request matcher
        +
An ordered list of security filters
```

Conceptually, the contract looks like this:

```
public interface SecurityFilterChain {

    boolean matches(HttpServletRequest request);

    List<Filter> getFilters();
}
```

For example:

```
SecurityFilterChain 1

Matcher:
    /api/**

Filters:
    SecurityContextHolderFilter
    BearerTokenAuthenticationFilter
    ExceptionTranslationFilter
    AuthorizationFilter
```

Another chain may be configured as:

```
SecurityFilterChain 2

Matcher:
    every request

Filters:
    SecurityContextHolderFilter
    CsrfFilter
    UsernamePasswordAuthenticationFilter
    BasicAuthenticationFilter
    AnonymousAuthenticationFilter
```

```
    ExceptionTranslationFilter
    AuthorizationFilter
```

The precise list depends on the features enabled through `HttpSecurity`.

Request matching

A `RequestMatcher` determines whether a chain applies to the current request. It can inspect properties such as:

- request path;
- HTTP method;
- headers;
- servlet path;
- parameters;
- dispatcher type.

For the request:

```
GET /api/users
```

the matcher `/api/**` returns `true`, while `/admin/**` returns `false`.

First matching chain wins

`FilterChainProxy` invokes only the first `SecurityFilterChain` that matches the request.

Suppose the chains are ordered as:

```
Chain 1: /api/**
Chain 2: /**
```

For `/api/users`, both patterns could match. Spring Security selects Chain 1 and stops searching. It does not combine the filters from both chains.

Therefore, chains should generally be ordered from the most specific matcher to the most general matcher.

```
1. /api/**
2. /admin/**
3. /**
```

If no chain matches, the request is not protected by a Spring Security filter chain. A fallback chain is therefore useful.

```
@Bean
SecurityFilterChain fallback(HttpSecurity http) throws Except
ion {
    http.authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
    );

    return http.build();
}
```

If no `securityMatcher(...)` is configured, the chain normally applies to every request.

---

5. `HttpSecurity`

`HttpSecurity` is a builder used to describe and construct a `SecurityFilterChain`.

```
http
    .csrf(Customizer.withDefaults())
    .formLogin(Customizer.withDefaults())
    .httpBasic(Customizer.withDefaults())
    .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
    );
```

Each configuration method contributes to the final chain:

```
http.csrf(...)
    ↓
Configure CSRF protection

http.formLogin(...)
    ↓
Configure form-login filters and handlers

http.httpBasic(...)
    ↓
Configure HTTP Basic authentication

http.authorizeHttpRequests(...)
    ↓
Configure authorization rules
```

`HttpSecurity` describes:

- which security features are enabled;
- which filters are required;
- how those filters are configured;
- the order in which they execute;
- which authentication providers they use;
- which authorization rules apply.

Finally:

```
return http.build();
```

converts the builder configuration into a concrete `SecurityFilterChain`.

---

6. Authentication Filters

Different authentication mechanisms receive credentials in different forms.

Form login

```
POST /login
Content-Type: application/x-www-form-urlencoded

username=aditya&password=secret123
```

This request is handled by `UsernamePasswordAuthenticationFilter`.

HTTP Basic

```
Authorization: Basic YWRpdHlhOnNlY3JldDEyMw==
```

This request is handled by `BasicAuthenticationFilter`.

A filter does not authenticate every request

`UsernamePasswordAuthenticationFilter` normally looks for a form-login request such as:

```
POST /login
```

For an ordinary request such as:

```
GET /courses
```

the filter does not treat it as a new form-login attempt. It continues the chain.

```
chain.doFilter(request, response);
```

Therefore:

```
Authentication filter present in the chain
        ≠
Authentication attempt performed on every request
```

The filter first checks whether the request contains the authentication mechanism it understands.

7. `Authentication` and the Request Token

The HTTP request initially contains plain values:

```
username = aditya
password = secret123
```

Spring Security converts them into a standard authentication object. For username-and-password authentication, this is commonly a `UsernamePasswordAuthenticationToken`.

Before verification:

```
UsernamePasswordAuthenticationToken
├── principal = "aditya"
├── credentials = "secret123"
├── authorities = []
└── authenticated = false
```

At this stage, the token represents only a claim:

> Someone claims to be Aditya and presents `secret123` as proof.

It does not mean Spring Security has accepted the identity.

The unauthenticated token can be represented as:

```
UsernamePasswordAuthenticationToken token =
        UsernamePasswordAuthenticationToken.unauthenticated(
                username,
                password
        );
```

Here:

```
principal   = claimed identity
credentials = claimed proof
```

The authentication filter should not contain database or password-verification logic. Its main responsibilities are:

```
Extract credentials
        ↓
Create an Authentication request
        ↓
Send it to the authentication engine
```

---

8. `AuthenticationManager`

`AuthenticationManager` is the entry point into Spring Security's authentication engine.

Its central operation is:

```
public interface AuthenticationManager {

    Authentication authenticate(Authentication authenticatio
n)
            throws AuthenticationException;
}
```

It receives an unauthenticated request token and produces one of two outcomes:

```
Unauthenticated Authentication
        ↓
AuthenticationManager.authenticate(...)
        ↓
        ├── Success → Authenticated Authentication
        │
        └── Failure → AuthenticationException
```

`AuthenticationManager` is an interface. Its most common implementation is `ProviderManager`.

9. `ProviderManager` and `AuthenticationProvider`

An application can support multiple authentication mechanisms:

- database username and password;
- JWT;
- one-time passwords;
- LDAP;
- SAML;
- remember-me authentication;
- custom authentication mechanisms.

`ProviderManager` coordinates the available `AuthenticationProvider` objects.

```
ProviderManager
    ├── DaoAuthenticationProvider
    ├── JwtAuthenticationProvider
    ├── RememberMeAuthenticationProvider
    └── CustomOtpAuthenticationProvider
```

Each provider understands a particular authentication-token type. Conceptually, a provider exposes a check similar to:

```
boolean supports(Class<?> authenticationType);
```

For example:

```
DaoAuthenticationProvider
    → supports UsernamePasswordAuthenticationToken
```

```
JwtAuthenticationProvider
    → supports a JWT authentication token
```

When `ProviderManager` receives a `UsernamePasswordAuthenticationToken`, it finds a provider capable of authenticating that token.

```
AuthenticationManager
        ↓ coordinates
AuthenticationProvider
        ↓ performs mechanism-specific verification
```

For database-backed username-and-password authentication, the relevant provider is `DaoAuthenticationProvider`.

---

10. Connecting Our Database to Spring Security

Our application has its own `User` entity. Spring Security does not know its structure and should not depend directly on our persistence model.

Spring Security therefore uses two contracts:

```
UserDetails
    → Spring Security's view of one user

UserDetailsService
    → Strategy for finding that user
```

Our `User` entity

The application may care about fields such as:

```
id
username
password
email
firstName
lastName
```

```
profilePicture
createdAt
subscription
preferences
roles
```

Spring Security mainly needs:

- username;
- stored password;
- authorities;
- enabled state;
- account-expiration state;
- account-lock state;
- credential-expiration state.

---

11. `UserDetails`

`UserDetails` represents Spring Security's view of one user.

Conceptually:

```java
public interface UserDetails {

    String getUsername();

    String getPassword();

    Collection<? extends GrantedAuthority> getAuthorities();

    boolean isAccountNonExpired();

    boolean isAccountNonLocked();

    boolean isCredentialsNonExpired();
```

```
    boolean isEnabled();
}
```

Spring Security is effectively saying:

> Store users however you want, but provide an object that satisfies this contract.

Two different `User` classes

An application may contain:

```
com.coderarmy.entity.User
```

Spring Security also provides:

```
org.springframework.security.core.userdetails.User
```

They are not the same class.

```
Application User
    → Database entity and domain model

Spring Security User
    → Built-in UserDetails implementation
```

We do not have to use Spring Security's concrete `User` class. A custom adapter often makes the bridge easier to understand.

`CustomUserDetails`

```
public class CustomUserDetails implements UserDetails {

    private final User user;

    public CustomUserDetails(User user) {
```

```
        this.user = user;
    }

    @Override
    public String getUsername() {
        return user.getUsername();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorit
ies() {
        return user.getRoles()
                .stream()
                .map(role -> new SimpleGrantedAuthority(role.
getName()))
                .toList();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
```

```
    }

    @Override
    public boolean isEnabled() {
        return user.isEnabled();
    }
}
```

This class adapts our database entity to the interface Spring Security understands.

---

13. `UserDetailsService`

`UserDetailsService` answers one central question:

> Given a username, how should Spring Security load that user?

Its main method is:

```
UserDetails loadUserByUsername(String username);
```

The distinction is simple:

```
UserDetails
    → One user

UserDetailsService
    → How to find that user
```

Or:

```
UserDetailsService = Finder
UserDetails        = Result
```

`CustomUserDetailsService`

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsS
ervice {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository
                .findByUsername(username)
                .orElseThrow(() ->
                        new UsernameNotFoundException("User n
ot found")
                );

        return new CustomUserDetails(user);
    }
}
```

The final line means:

```
Convert our application's User
        ↓
into Spring Security's UserDetails view
```

If email is used as the login identity, the same contract can query by email:

```
@Override
public UserDetails loadUserByUsername(String email) {
    User user = userRepository
            .findByEmail(email)
            .orElseThrow(() ->
                    new UsernameNotFoundException("User not f
ound")
```

```
            );

    return new CustomUserDetails(user);
}
```


Is `CustomUserDetails` compulsory?

No. We can directly create Spring Security's built-in implementation:

```
return org.springframework.security.core.userdetails.User
        .withUsername(user.getUsername())
        .password(user.getPassword())
        .authorities(
                user.getRoles()
                        .stream()
                        .map(role -> new SimpleGrantedAuthori
ty(role.getName()))
                        .toList()
        )
        .disabled(!user.isEnabled())
        .build();
```

The custom class is useful when we want the mapping to be explicit or need access to additional domain-user properties.

---

14. `DaoAuthenticationProvider`

At this point, we have a user-loading flow:

```
Spring Security
      ↓ username
UserDetailsService
      ↓
UserRepository
      ↓
Database User
      ↓
```

```
CustomUserDetails
      ↓
UserDetails
```

We still need a component that can:

1. receive the authentication request;
2. load the user through `UserDetailsService`;
3. obtain the stored encoded password;
4. compare it with the submitted raw password;
5. check the account state;
6. return a trusted authentication result.

That component is `DaoAuthenticationProvider`.

```
                DaoAuthenticationProvider
                        /       \
                       /         \
                      ↓           ↓
          UserDetailsService  PasswordEncoder
                    ↓             ↓
            UserRepository  BCryptPasswordEncoder
                    ↓
                Database
```

`DaoAuthenticationProvider` is an `AuthenticationProvider` designed for username-and-password authentication using a `UserDetailsService` and a `PasswordEncoder`.

DAO stands for Data Access Object. The name reflects that user information is retrieved from a user-data source before credentials are verified.

---

15. Complete Internal Login Flow

The full flow can now be understood as one sequence.

```
HTTP Request
    ↓
Authentication Filter
```

```
    ↓
Unauthenticated UsernamePasswordAuthenticationToken
    ↓
AuthenticationManager
    ↓
ProviderManager
    ↓
DaoAuthenticationProvider
    ├── UserDetailsService → Repository → Database
    └── PasswordEncoder → Password verification
    ↓
Authenticated UsernamePasswordAuthenticationToken
    ↓
SecurityContext
    ↓
SecurityContextHolder
    ↓
Authorization
    ↓
Controller
```

Step 1: The filter extracts the credentials

For form login, `UsernamePasswordAuthenticationFilter` reads the submitted username and password. For HTTP Basic, `BasicAuthenticationFilter` reads them from the `Authorization` header.

Step 2: An unauthenticated token is created

```
UsernamePasswordAuthenticationToken

principal     = "aditya"
credentials   = "secret123"
authorities   = []
authenticated = false
```

The filter calls:

```
authenticationManager.authenticate(authenticationToken);
```

Step 3: `ProviderManager` selects a provider

`ProviderManager` checks which configured provider supports `UsernamePasswordAuthenticationToken`.

```
ProviderManager
      ↓
DaoAuthenticationProvider supports the token
```

Step 4: The username is extracted

The provider reads the claimed identity:

```
principal = "aditya"
```

It then asks the `UserDetailsService` for trusted user data:

```
userDetailsService.loadUserByUsername("aditya");
```

Step 5: The database is queried

Our service uses the repository:

```
User user = userRepository
        .findByUsername(username)
        .orElseThrow(() ->
                new UsernameNotFoundException("User not foun
d")
        );

return new CustomUserDetails(user);
```

The provider now has trusted information from the database, including the encoded password and authorities.

Step 6: Account status is checked

Spring Security evaluates values exposed by `UserDetails`:

```
isEnabled()
isAccountNonLocked()
isAccountNonExpired()
isCredentialsNonExpired()
```

For example, an account with:

```
enabled = false
```

cannot authenticate even if the submitted password is correct.

Step 7: The password is verified

The provider has two different values:

```
Login request:
    secret123

Database:
    $2a$10$...
```

It delegates password verification to the configured `PasswordEncoder`:

```
passwordEncoder.matches(
        rawPassword,
        userDetails.getPassword()
);
```

Conceptually:

```
secret123
    +
$2a$10$...
    ↓
BCryptPasswordEncoder.matches(...)
```

```
    ↓
true or false
```

The raw password is not encoded again and compared as a string. `matches(...)` performs the verification using the information stored in the BCrypt representation.

Step 8: A successful authentication result is created

If the user exists, the account is valid, and the password matches, the provider returns a trusted authentication object.

```
UsernamePasswordAuthenticationToken

principal:
    CustomUserDetails

credentials:
    erased or null

authorities:
    ROLE_USER

authenticated:
    true
```

The object has changed from an unverified claim into a verified identity.

```
Claim
  ↓ Authentication
Verified identity
```

Sensitive credential information is normally erased after successful authentication so the raw password is not retained longer than necessary.

Step 9: Authentication is stored in the security context

The successful result is placed into the current security context.

```
Authentication
      ↓
SecurityContext
      ↓
SecurityContextHolder
```

`SecurityContextHolder` provides access to the current request's authenticated security context.

Step 10: Authorization and controller execution continue

Spring Security can now evaluate rules such as:

```
.requestMatchers("/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

If access is allowed, the request reaches the controller. The controller does not need to query the user or verify the password again.

---

16. What Happens When Authentication Fails?

User not found

```
throw new UsernameNotFoundException("User not found");
```

Wrong password

```
passwordEncoder.matches(
        "wrongPassword",
        storedEncodedPassword
);
```

returns `false`. Authentication fails, commonly through a `BadCredentialsException`.

Invalid account state

Authentication can also fail when the account is:

- disabled;
- locked;
- expired;
- using expired credentials.

In each case, no trusted authentication result is placed into the security context.

---

17. Form Login and HTTP Basic Share the Same Backend

Form login and HTTP Basic receive credentials differently, but both can use the same database-authentication engine.

```
Form Login
    ↓
UsernamePasswordAuthenticationFilter
    ↓
UsernamePasswordAuthenticationToken
    ↓
AuthenticationManager
    ↓
DaoAuthenticationProvider
    ↓
Database authentication
```

```
HTTP Basic
    ↓
BasicAuthenticationFilter
    ↓
UsernamePasswordAuthenticationToken
    ↓
AuthenticationManager
    ↓
DaoAuthenticationProvider
```

```
    ↓
Database authentication
```

The filters are different because the HTTP credential formats are different. The underlying username-and-password verification process is the same.

18. Registration and Login Are Separate Flows

Registration stores a new user. Login verifies an existing user.

Registration flow

```
Client
  ↓ username + raw password
AuthService
  ↓
PasswordEncoder.encode(...)
  ↓
BCrypt encoded password
  ↓
UserRepository
  ↓
Database
```

Login flow

```
Client
  ↓ username + raw password
Authentication filter
  ↓
AuthenticationManager
  ↓
DaoAuthenticationProvider
  ├── Load stored user
  └── PasswordEncoder.matches(...)
  ↓
Authenticated Authentication
```

Registration uses:

```
passwordEncoder.encode(rawPassword);
```

Login uses:

```
passwordEncoder.matches(rawPassword, storedEncodedPassword);
```

The server should assign the default role

A normal registration request should not be allowed to choose an administrative role.

Unsafe design:

```
{
  "username": "hacker",
  "password": "123",
  "role": "ROLE_ADMIN"
}
```

Instead, the server chooses the default role:

```
Role role = roleRepository
        .findByName("ROLE_USER")
        .orElseThrow();

user.getRoles().add(role);
```

The registration endpoint should also return a response DTO rather than the entity containing the stored password.

---

19. Project Structure

A possible structure is:

```
entity/
    User.java
    Role.java

repository/
    UserRepository.java
    RoleRepository.java

dto/
    RegisterRequest.java
    UserResponse.java

security/
    CustomUserDetails.java
    CustomUserDetailsService.java

service/
    AuthService.java

controller/
    AuthController.java
    TestController.java

config/
    SecurityConfig.java
```

20. Complete Implementation

`User` entity

```java
@Entity
@Table(name = "users")
@Getter
@Setter
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    private boolean enabled = true;

    @ManyToMany
    @JoinTable(
            name = "user_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_i
d")
    )
    private Set<Role> roles = new HashSet<>();
}
```

`Role` entity

```
@Entity
@Table(name = "roles")
@Getter
@Setter
@NoArgsConstructor
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
```

```
    private String name;
}
```

Role names can be stored as:

```
ROLE_USER
ROLE_ADMIN
```

When using:

```
.hasRole("ADMIN")
```

Spring Security checks for the authority `ROLE_ADMIN`.

Repositories

```
public interface UserRepository extends JpaRepository<User, L
ong> {

    Optional<User> findByUsername(String username);
}
```

```
public interface RoleRepository extends JpaRepository<Role, L
ong> {

    Optional<Role> findByName(String name);
}
```

DTOs

```
public record RegisterRequest(
        String username,
```

```
        String password
) {}
```

```
public record UserResponse(
        Long id,
        String username
) {}
```

CustomUserDetails

```
public class CustomUserDetails implements UserDetails {

    private final User user;

    public CustomUserDetails(User user) {
        this.user = user;
    }

    @Override
    public String getUsername() {
        return user.getUsername();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorit
ies() {
        return user.getRoles()
                .stream()
                .map(role -> new SimpleGrantedAuthority(role.
```

```
getName()))
                .toList();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return user.isEnabled();
    }
}
```

CustomUserDetailsService

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsS
ervice {

    private final UserRepository userRepository;

    @Override
```

```
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository
                .findByUsername(username)
                .orElseThrow(() ->
                        new UsernameNotFoundException("User n
ot found")
                );

        return new CustomUserDetails(user);
    }
}
```

`PasswordEncoder` bean

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Visible `DaoAuthenticationProvider` bean

Spring Security can construct much of the username-and-password infrastructure automatically when the required beans are available. For learning the internal architecture, explicitly creating the provider makes the dependencies visible.

```
@Bean
public DaoAuthenticationProvider authenticationProvider(
        CustomUserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder
) {
    DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userDetailsServic
e);
```

```
        provider.setPasswordEncoder(passwordEncoder);

        return provider;
    }
```

This configuration directly shows:

```
DaoAuthenticationProvider
    ├── CustomUserDetailsService
    └── PasswordEncoder
```

SecurityConfig

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider(
            CustomUserDetailsService userDetailsService,
            PasswordEncoder passwordEncoder
    ) {
        DaoAuthenticationProvider provider =
                new DaoAuthenticationProvider(userDetailsServ
ice);

        provider.setPasswordEncoder(passwordEncoder);

        return provider;
    }
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http,
            DaoAuthenticationProvider provider
    ) throws Exception {

        http
            // Disabled only for the current classroom API de
mo.
            .csrf(csrf -> csrf.disable())

            .authenticationProvider(provider)

            .authorizeHttpRequests(auth -> auth
                    .requestMatchers("/register").permitAll()
                    .requestMatchers("/admin/**").hasRole("AD
MIN")
                    .anyRequest().authenticated()
            )

            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Registration service

```
@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
```

```
    private final PasswordEncoder passwordEncoder;

    public UserResponse register(RegisterRequest request) {
        User user = new User();

        user.setUsername(request.username());

        user.setPassword(
                passwordEncoder.encode(request.password())
        );

        user.setEnabled(true);

        Role role = roleRepository
                .findByName("ROLE_USER")
                .orElseThrow(() ->
                        new IllegalStateException("ROLE_USER
not found")
                );

        user.getRoles().add(role);

        User saved = userRepository.save(user);

        return new UserResponse(
                saved.getId(),
                saved.getUsername()
        );
    }
}
```

Registration controller

```
@RestController
@RequiredArgsConstructor
```

```
public class AuthController {

    private final AuthService authService;

    @PostMapping("/register")
    public UserResponse register(
            @RequestBody RegisterRequest request
    ) {
        return authService.register(request);
    }
}
```

21. CSRF and the Postman Demo

When CSRF protection is enabled, a state-changing request such as:

```
POST /register
```

can be rejected if Postman does not send a valid CSRF token.

For a short classroom API demonstration focused only on authentication, CSRF may be temporarily disabled:

```
.csrf(csrf -> csrf.disable())
```

> Disabling CSRF is a demo decision, not a universal production-security recommendation. The correct production configuration depends on how clients authenticate and whether browsers automatically attach credentials.

22. Testing an Authenticated Endpoint

Simple protected endpoint

```
@RestController
public class TestController {

    @GetMapping("/profile")
    public String profile() {
        return "Welcome to your profile";
    }
}
```

Because the configuration contains:

```
.anyRequest().authenticated()
```

`/profile` requires authentication.

Display the current `Authentication`

```
@GetMapping("/me")
public Map<String, Object> me(Authentication authentication)
{
    return Map.of(
            "username", authentication.getName(),
            "authorities", authentication.getAuthorities(),
            "authenticated", authentication.isAuthenticated()
    );
}
```

After successful authentication, the response may look like:

```
{
  "username": "aditya",
  "authorities": [
    {
      "authority": "ROLE_USER"
    }
```

```
  ],
  "authenticated": true
}
```

This endpoint makes the final authenticated object visible during the demonstration.

---

23. Suggested Demonstration Sequence

1. Create the default role

Ensure the `roles` table contains:

```
ROLE_USER
```

2. Register a user

```
POST /register
Content-Type: application/json

{
  "username": "aditya",
  "password": "secret123"
}
```

Show that the database contains an encoded password rather than the raw password.

3. Call a protected endpoint without credentials

```
GET /me
```

The request should not receive protected data.

4. Call it with correct HTTP Basic credentials

```
Username: aditya
Password: secret123
```

The endpoint should return the authenticated username and authorities.

5. Try a wrong password

The authentication attempt should fail even though the user exists.

6. Disable the account

Set:

```
enabled = false
```

Authentication should now fail even with the correct password.

This demonstrates that database authentication is more than password comparison; account state is also part of the decision.

---

24. Final Mental Model

The complete architecture can be remembered in four layers.

Layer 1: Receive credentials

```
UsernamePasswordAuthenticationFilter
or
BasicAuthenticationFilter
```

Layer 2: Represent the request

```
UsernamePasswordAuthenticationToken
```

Layer 3: Authenticate the request

```
AuthenticationManager
    ↓
ProviderManager
    ↓
DaoAuthenticationProvider
```

Layer 4: Load and verify the user

```
UserDetailsService → UserRepository → Database

PasswordEncoder → Password verification
```

After success:

```
Authenticated Authentication
        ↓
SecurityContext
        ↓
SecurityContextHolder
        ↓
Authorization
        ↓
Controller
```

Key Takeaways

1. Spring Security operates through servlet filters before the controller.
2. `DelegatingFilterProxy` bridges Tomcat and Spring-managed security components.
3. `FilterChainProxy` selects the first matching `SecurityFilterChain`.

4. `HttpSecurity` builds and configures that chain.
5. Authentication filters extract credentials; they do not perform database verification themselves.
6. `UsernamePasswordAuthenticationToken` represents both the unverified request and, later, the verified result.
7. `AuthenticationManager` is the entry point into the authentication engine.
8. `ProviderManager` delegates to a compatible `AuthenticationProvider`.
9. `DaoAuthenticationProvider` connects `UserDetailsService` with `PasswordEncoder`.
10. `UserDetails` represents one user; `UserDetailsService` explains how to find that user.
11. Registration uses `encode(...)`; login uses `matches(...)`.
12. After successful login, the authenticated object is placed in the `SecurityContextHolder` and authorization continues.